



SCHOOL OF INFORMATICS & IT
Project Solution (Individual)

Student Name (Matric Number) : Mohamed Tawfeeq S/O Mohamed Harish(2400212C)

Tutorial Group : TC18

Tutor : Hanjin NG

Submission Date : 08/08/2025

Declaration of Originality

I am the originator of this work and I have appropriately acknowledged all other original sources used as my references for this work.

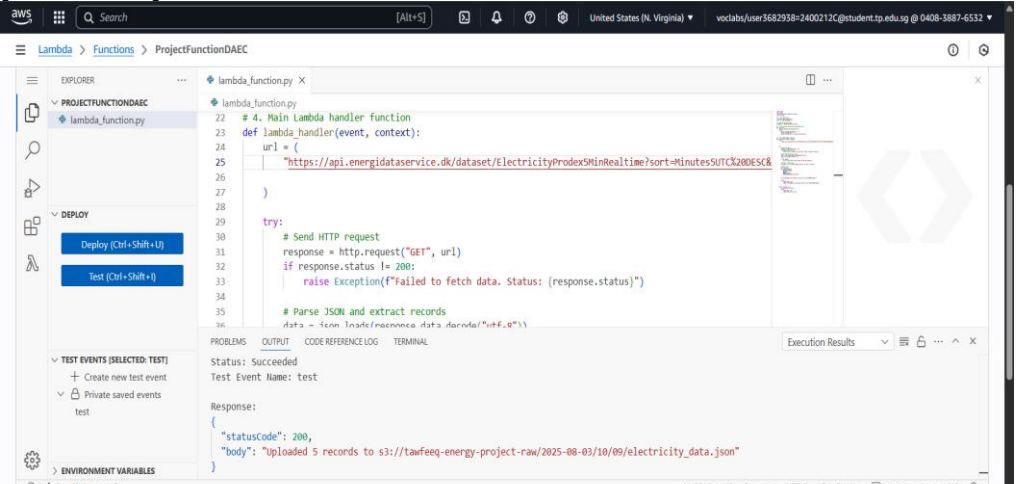
I understand that Plagiarism is the act of taking and using the whole or any part of another person's work, including work generated by AI, and presenting it as my own.

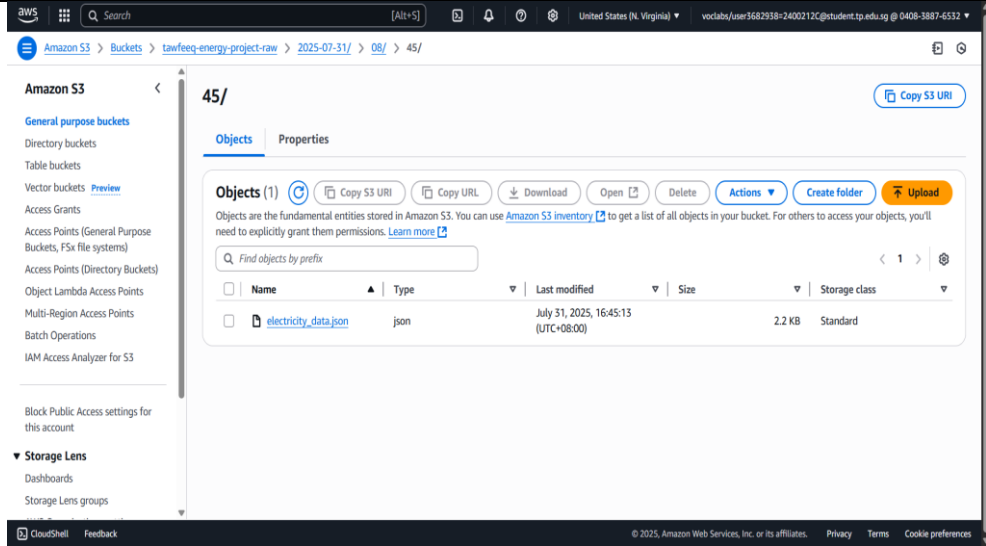
I understand that Plagiarism is an academic offence and if I am found to have committed or abetted the offence of plagiarism in relation to this submitted work, disciplinary action will be enforced.

Project Solution

Basic Requirements

1. Data Ingestion

Lambda	Setup: I created a function called “ProjectFunctionDAEC”. I used python 3.13 and the functions call the Energinet API to retrieve real time data. The last 5 rows will be retrieved in JSON into a structured format. This data will be stored in the raw S3 bucket. The python code involves the name of my S3 bucket, so it recognises where to ingest the data. When deployed and test events are run, it successfully uploads the data into the correct bucket. The data is partitioned like this for the raw bucket: 1. tawfeeq-energy-project-raw 2. 2025-07-31/ 3. 08/ 4. 45/ 5. electricity_data.json This is optimized for glue crawlers and Athena queries.
	[screenshot]  The screenshot displays the AWS Lambda console interface. The left sidebar shows the 'Functions' section with 'ProjectFunctionDAEC' selected. The main area shows the Python code for the lambda function. The code defines a 'lambda_handler' function that sends an HTTP GET request to the Energinet API, parses the JSON response, and uploads the data to an S3 bucket. The 'Execution Results' panel at the bottom shows a successful status with a test event name 'test' and a response body indicating that 5 records were uploaded to the specified S3 path. <pre>22 # 4. Main Lambda handler function 23 def lambda_handler(event, context): 24 url = (25 "https://api.energinetdata.dk/dataset/ElectricityProdx5MinRealtime?sort=MinutesUTC&offset=0" 26) 27 28 try: 29 # Send HTTP request 30 response = http.request("GET", url) 31 if response.status != 200: 32 raise Exception(f"Failed to fetch data. Status: {response.status}") 33 34 # Parse JSON and extract records 35 data = json.loads(response.data.decode("utf-8")) 36 37 except Exception as e: 38 print(f"Error: {e}") 39 return {"statusCode": 500, "body": str(e)}</pre> Execution Results Status: Succeeded Test Event Name: test Response: <pre>{ "statusCode": 200, "body": "Uploaded 5 records to s3://tawfeeq-energy-project-raw/2025-08-03/10/09/electricity_data.json" }</pre>

	
	Justification: Lambda is a serverless and a cost-effective platform designed for tasks such as API ingestion. A limit was set to 5 rows for every 5 minutes as this is to ensure we only get the new data and to avoid duplicates. If an error were to occur, the system will send the log details to CloudWatch Logs which can be monitored in real-time. Cost Consideration: Per day, there can be around 288 API requests and around 8,600 request per month. This results in a total cost of only around \$0.20 per month which is relatively cheap.

2. Data Storage

S3	Setup: My S3 has 3 buckets. One for raw data (tawfeeq-energy-project-raw), one for transformed data (tawfeeq-energy-project-transform), one for Athena query (athenaquerydaec). The purpose of the raw bucket is storing raw JSON data coming in from the Lambda API ingestion. The transformed bucket contains transformed data in CSV format after processing in ETL job. The Athena bucket is a storage for the Athena queries later on. Configuration: AWS KMS was used for encryption using KMS keys to protect the data integrity. Versioning was used on the transformed bucket to have rollback of the data in case the data has any errors such as corruption during the transformation process. This is so that we can get the data back. Lifecycle rules were used for Athena Query Results in the S3 bucket to automatically delete the outdated data after 7 days to minimize storage cost. Public access was blocked to ensure data integrity, and that the data cannot be access by the public, giving access to only authorized users. [screenshot]
----	---

Amazon S3

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access Grants

Access Points (General Purpose Buckets, FSx file systems)

Access Points (Directory Buckets)

Object Lambda Access Points

Multi-Region Access Points

Batch Operations

IAM Access Analyzer for S3

Block Public Access settings for this account

Storage Lens

Dashboards

Storage Lens groups

45/

Objects

Properties

Objects (1)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

Name	Type	Last modified	Size	Storage class
electricity_data.json	json	July 31, 2025, 16:45:13 (UTC+08:00)	2.2 KB	Standard

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Amazon S3

Buckets

athenaquerydaec

Lifecycle configuration

Athena Lifecycle

Lifecycle rule configuration

Lifecycle rule name

Athena Lifecycle

Status

Enabled

Scope

Filtered

Prefix

t

Object tags

-

Minimum object size

-

When no minimum object size is specified, the minimum object size for transitions is determined by the lifecycle configuration.

Maximum object size

-

Review transition and expiration actions

Current version actions

Day 0

Objects uploaded

Day 7

Objects expire

Noncurrent versions actions

Day 0

No actions defined.

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Amazon S3

Buckets

lawfeeq-energy-project-transform

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures.

Bucket Versioning

Enabled

Multi-factor authentication (MFA) delete

Disabled

Tags (0)

You can use bucket tags to track storage costs and organize buckets.

Key

Value

No tags associated with this resource.

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type

Server-side encryption with AWS Key Management Service keys (SSE-KMS)

Encryption key ARN

arn:aws:kms:us-east-1:4088886532:key/127465c-a166-489c-a4b3-60481f72b2d5

Bucket Key

When KMS encryption is used to encrypt new objects in this bucket, the bucket key reduces encryption costs by lowering calls to AWS KMS.

Enabled

Intelligent-Tiering Archive configurations (0)

View details

Edit

Delete

Create configuration

	General purpose buckets (9) Info Empty Delete Create bucket Buckets are containers for data stored in S3. Find buckets by name < 1 > <table><tr><th></th><th>Name ▲</th><th>AWS Region ▼</th><th>Creation date ▼</th></tr><tr><td>☐</td><td>athenaquerydaec</td><td>US East (N. Virginia) us-east-1</td><td>July 31, 2025, 17:02:22 (UTC+08:00)</td></tr><tr><td>☐</td><td>aws-cloudtrail-logs-040838876532-9ae4144e</td><td>US East (N. Virginia) us-east-1</td><td>July 31, 2025, 14:24:56 (UTC+08:00)</td></tr><tr><td>☐</td><td>aws-glue-assets-040838876532-us-east-1</td><td>US East (N. Virginia) us-east-1</td><td>July 26, 2025, 13:45:01 (UTC+08:00)</td></tr><tr><td>☐</td><td>calabs-bucket-tawfeeq02</td><td>US West (Oregon) us-west-2</td><td>July 10, 2025, 14:33:51 (UTC+08:00)</td></tr><tr><td>☐</td><td>daec-test-2400212c</td><td>US East (N. Virginia) us-east-1</td><td>June 10, 2025, 14:17:58 (UTC+08:00)</td></tr><tr><td>☐</td><td>projecttawfeeqdaec02</td><td>US East (N. Virginia) us-east-1</td><td>July 24, 2025, 15:22:56 (UTC+08:00)</td></tr><tr><td>☐</td><td>tawfeeq-energy-project-2025</td><td>US East (N. Virginia) us-east-1</td><td>July 26, 2025, 11:38:50 (UTC+08:00)</td></tr><tr><td>☐</td><td>tawfeeq-energy-project-raw</td><td>US East (N. Virginia) us-east-1</td><td>July 26, 2025, 13:23:12 (UTC+08:00)</td></tr><tr><td>☐</td><td>tawfeeq-energy-project-transform</td><td>US East (N. Virginia) us-east-1</td><td>July 26, 2025, 13:17:41 (UTC+08:00)</td></tr></table>		Name ▲	AWS Region ▼	Creation date ▼	☐	athenaquerydaec	US East (N. Virginia) us-east-1	July 31, 2025, 17:02:22 (UTC+08:00)	☐	aws-cloudtrail-logs-040838876532-9ae4144e	US East (N. Virginia) us-east-1	July 31, 2025, 14:24:56 (UTC+08:00)	☐	aws-glue-assets-040838876532-us-east-1	US East (N. Virginia) us-east-1	July 26, 2025, 13:45:01 (UTC+08:00)	☐	calabs-bucket-tawfeeq02	US West (Oregon) us-west-2	July 10, 2025, 14:33:51 (UTC+08:00)	☐	daec-test-2400212c	US East (N. Virginia) us-east-1	June 10, 2025, 14:17:58 (UTC+08:00)	☐	projecttawfeeqdaec02	US East (N. Virginia) us-east-1	July 24, 2025, 15:22:56 (UTC+08:00)	☐	tawfeeq-energy-project-2025	US East (N. Virginia) us-east-1	July 26, 2025, 11:38:50 (UTC+08:00)	☐	tawfeeq-energy-project-raw	US East (N. Virginia) us-east-1	July 26, 2025, 13:23:12 (UTC+08:00)	☐	tawfeeq-energy-project-transform	US East (N. Virginia) us-east-1	July 26, 2025, 13:17:41 (UTC+08:00)
	Name ▲	AWS Region ▼	Creation date ▼																																						
☐	athenaquerydaec	US East (N. Virginia) us-east-1	July 31, 2025, 17:02:22 (UTC+08:00)																																						
☐	aws-cloudtrail-logs-040838876532-9ae4144e	US East (N. Virginia) us-east-1	July 31, 2025, 14:24:56 (UTC+08:00)																																						
☐	aws-glue-assets-040838876532-us-east-1	US East (N. Virginia) us-east-1	July 26, 2025, 13:45:01 (UTC+08:00)																																						
☐	calabs-bucket-tawfeeq02	US West (Oregon) us-west-2	July 10, 2025, 14:33:51 (UTC+08:00)																																						
☐	daec-test-2400212c	US East (N. Virginia) us-east-1	June 10, 2025, 14:17:58 (UTC+08:00)																																						
☐	projecttawfeeqdaec02	US East (N. Virginia) us-east-1	July 24, 2025, 15:22:56 (UTC+08:00)																																						
☐	tawfeeq-energy-project-2025	US East (N. Virginia) us-east-1	July 26, 2025, 11:38:50 (UTC+08:00)																																						
☐	tawfeeq-energy-project-raw	US East (N. Virginia) us-east-1	July 26, 2025, 13:23:12 (UTC+08:00)																																						
☐	tawfeeq-energy-project-transform	US East (N. Virginia) us-east-1	July 26, 2025, 13:17:41 (UTC+08:00)																																						
	Justification: S3 was used as it is highly scalable and fully integrated with services such as Glue, Athena and Lambda which are used as part of the data pipeline. Separate buckets were used for raw and transform to separate them accordingly and in case of any debugging. KMS encryption ensure strong data security and versioning will protect against and accidental overwrites. Lifecycle rules will control the costs making it cost-effective as it will reduce excessive cost on storage. For better performance in Athena, the data is partitioned by the date and allows Athena to only scan the relevant partitions Cost Consideration: The JSON and CSV formatted files are small resulting in low cost. Lifecycle Rules reduces costs further by deleting the old query results. Versioning is only enabled where necessary such as when it undergoes transformation process to avoid excessive costs. It is estimated to have a low cost of \$0.10 which is very budget friendly.																																								

3. Data Process

AW S Glue	Setup: I first created two databases. “Raw data” database was created to store the raw data coming in from the raw s3 bucket. “Transform data” database was created to store the transformed data to be used for Athena query later. The first crawler was created (“Crawl Raw Data New”) to fetch the data from the S3 bucket and place it into a table. The data here will be used in the ETL job to process the data and convert it into tabular format, CSV. The ETL job will also drop and duplicates in the data if there is any. Once all the processing is done, the data will be stored in the transformed S3 bucket. This is where the second crawler comes in to crawl the transformed bucket and place it into a table in the “Transform data” database. Job bookmarks was also enabled to prevent reprocessing data that has already been processed.
	[screenshot]

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations [New](#)

▼ Data Catalog

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

► Data Integration and ETL

► Legacy pages

What's New [↗](#)

Documentation [↗](#)

AWS Marketplace

☐ Enable compact mode

☐ Enable new navigation

CrawlRawDataNew

Last updated (UTC)
August 4, 2025 at 02:26:31

Run crawlerEditDelete

Crawler properties

Name

CrawlRawDataNew

IAM role

[LabRole](#)

Database

rawdata

State

READY

Description

-

Security configuration

-

Lake Formation configuration

-

Table prefix

-

Maximum table threshold

-

► Advanced settings

Crawler runs

Schedule

Data sources

Classifiers

Tags

Crawler runs (1)

The list of crawler runs for this crawler.

Filter data

Filter by a date and time range

Stop runView CloudWatch logsView run details

Start time (UTC)

End time (UTC)

Current/last duration

Status

DPU hours

Table changes

July 31, 2025 at 08:49:32

July 31, 2025 at 08:50:42

01 min 10 s

Completed

0.039

1 table change, 1 partition change

CloudShellFeedback

© 2025, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations [New](#)

▼ Data Catalog

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

► Data Integration and ETL

► Legacy pages

What's New [↗](#)

Documentation [↗](#)

AWS Marketplace

☐ Enable compact mode

☐ Enable new navigation

Schema

Partitions

Indexes

Column statistics - new

Schema (18)

View and manage the table schema.

Edit schema as JSONEdit schema

Filter schemas

#

Column name

Data type

Partition key

Comment

1

minutes5utc

string

-

-

2

minutes5dk

string

-

-

3

pricesna

string

-

-

4

productionit100mw

double

-

-

5

productionge100mw

double

-

-

6

offshorewindpower

double

-

-

7

onshorewindpower

double

-

-

8

solarpower

double

-

-

9

exchangegrauvelt

double

-

-

10

exchangegermany

double

-

-

11

exchangenertherlands

double

-

-

12

exchangegreatbritain

double

-

-

13

exchangenorway

double

-

-

14

exchangesweden

double

-

-

15

bomholme4

double

-

-

16

partition_0

string

Partition (0)

-

17

partition_1

string

Partition (1)

-

18

partition_2

string

Partition (2)

-

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations [New](#)

▼ Data Catalog

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

► Data Integration and ETL

► Legacy pages

What's New [↗](#)

Documentation [↗](#)

AWS Marketplace

☐ Enable compact mode

☐ Enable new navigation

ETL JOB Proj

Last modified on 31/07/2025, 16:56:49

ActionsSaveRun

Visual

Script

Job details

Runs

Data quality

Schedules

Version Control

Data source - Data Catalog

AWS Glue Data Catalog

Transform - DropDuplicates

Drop Duplicates

Transform - DropNullFields

Drop Null Fields

Data target - S3 bucket

Amazon S3

Unsaved job found

We found an unsaved job, do you wish to restore it?

Restore

Job bookmark [Info](#)

Specifies how AWS Glue processes job bookmark when the job runs. It can remember previously processed data (Enable), update state information (Pause), or ignore state information (Disable).

Enable

Tables (5)

Last updated (UTC)
August 4, 2025 at 07:23:42

Delete

Add tables using crawler

Add table

View and manage all available tables.

Q Filter tables

< 1 >

☐	Name	Database	Location	Classific...	Depreca...	View data	Data quality	Column statistics
☐	cloudtrail_logs	database_24002	s3://aws-cloudu	-	-	Table data	View data qualit	View statistics
☐	daec_test_24002	database_24002	s3://daec-test-2	CSV	-	Table data	View data qualit	View statistics
☐	tawfeeenergy	rawdata	s3://tawfeeenergy	JSON	-	Table data	View data qualit	View statistics
☐	tawfeeenergy	raw_data_db	s3://tawfeeenergy	JSON	-	Table data	View data qualit	View statistics
☐	tawfeeenergy	transformdata	s3://tawfeeenergy	CSV	-	Table data	View data qualit	View statistics

tawfeeenergy-project-transform

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (2)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Q Find objects by prefix

Show versions

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	cleaned/	Folder	-	-	-
☐	run- AmazonS3_node1753952034387-1-part-r-00000	-	July 31, 2025, 16:58:48 (UTC+08:00)	1.1 KB	Standard

Justification:

Two Glue databases were used to separate them based off the raw and transformed data tables. This ensures they can be called for any processing or data querying without any confusion. Glue crawlers were used as they ensure that metadata is always up-to-date and accessible for querying. They automate the process for crawling and pulling the data from the S3 bucket. ETL job was used to clean and structure the raw data into an analytical format easier for querying (CSV). It also ensures that duplicate fields will be removed before data analysis, making it even more suitable for querying. CSV format was used as it is widely supported by Athena and it makes it an efficient choice for transforming and querying datasets.

Cost Considerations:

Using 2 glue crawlers per month would cost only around \$0.10, ETL job per execution will only cost \$0.25, making this component suitable and cost efficient as part of the data pipeline.

4. Data Serving

Athena	Setup: After the Glue Crawler has placed the transformed data into the database, Athena can query the table with data from the “Transform Data” database. The SQL queries will be stored in the Athena Query S3 bucket in csv format. The first query was used to display the first 10 rows of the dataset using the LIMIT function. The second query was used to calculate the total energy production by region. The third query was used to compare offshore wind production with onshore wind production for DK1 and DK2. The last query was used to calculate the average solar power production for each region. These queries helped to get better insights of the energy production trends.. [screenshot]
--------	--

Amazon Athena

Query editor tabs

transformdata

Tables and views

Create

Filter tables and views

Tables (1)

tawfeeq_energy_project_transform

Views (0)

SQL Ln 1, Col 1

Run again

Explain

Cancel

Clear

Create

Reuse query results up to 60 minutes ago

Query results

Query status

Completed

Time in queue: 107 ms

Run time: 583 ms

Data scanned: 1.05 KB

Results (5)

Search rows

minutesSutc minutesSdlk pricearea production100mw production100mw offshorewindpower onshorewindpower solarpower exch

1 2025-07-31T08:35:00 2025-07-31T10:35:00 DK2 131.610001 114.870003 135.070007 38.16 521.5 61.5

2 2025-07-31T08:30:00 2025-07-31T10:30:00 DK1 176.529999 1.81 305.540009 127.629997 1489.670044 -68.1

3 2025-07-31T08:40:00 2025-07-31T10:40:00 DK1 175.669998 1.76 308.339985 129.199997 1480.040039 -60.1

4 2025-07-31T08:35:00 2025-07-31T10:35:00 DK1 175.190002 2.02 306.970001 132.080002 1555.359985 -61.1

5 2025-07-31T08:40:00 2025-07-31T10:40:00 DK2 130.070007 121.580002 130.100006 41.330002 529.349976 60.5

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

United States (N. Virginia)

wsclshs/user36A2938-2402112C@student.tp.edu.sg @ 0406-3887-4332

Amazon Athena

Query editor tabs

Data

Data source

AwsDataCatalog

Catalogue

None

Database

transformdata

Tables and views

Create

Filter tables and views

Tables (1)

tawfeeq_energy_project_transform

Views (0)

Query 1 Query 2 Query 4 Query 5 Query 6 Query 7 Query 8 Query 9 Query 10 Query 11

1 SELECT pricearea,

2 SUM(production100mw + production100mw + offshorewindpower + onshorewindpower + solarpower) AS total_energy_produced

3 FROM "transformdata"."tawfeeq_energy_project_transform"

4 WHERE minutesSutc = (SELECT MAX(minutesSutc) FROM "transformdata"."tawfeeq_energy_project_transform")

5 GROUP BY pricearea

6 ORDER BY total_energy_produced DESC;

7

SQL Ln 7, Col 1

Run again

Explain

Cancel

Clear

Create

Reuse query results up to 60 minutes ago

Query results

Query status

Completed

Time in queue: 73 ms

Run time: 1.215 sec

Data scanned: 4.26 KB

Results (2)

Search rows

pricearea total_energy_produced

1 DK1 4190.060038

2 DK2 1904.859986

Amazon Athena

Query editor tabs

Data

Data source

AwsDataCatalog

Catalogue

None

Database

transformdata

Tables and views

Create

Filter tables and views

Tables (1)

tawfeeq_energy_project_transform

Views (0)

Query 1 Query 2 Query 4 Query 5 Query 6 Query 7 Query 8 Query 9 Query 10 Query 11

1 SELECT pricearea,

2 SUM(offshorewindpower) AS total_offshore_wind,

3 SUM(onshorewindpower) AS total_onshore_wind,

4 FROM "transformdata"."tawfeeq_energy_project_transform"

5 WHERE pricearea IN ('DK1', 'DK2')

6 GROUP BY pricearea

7 ORDER BY pricearea;

8

SQL Ln 8, Col 1

Run again

Explain

Cancel

Clear

Create

Reuse query results up to 60 minutes ago

Query results

Query status

Completed

Time in queue: 134 ms

Run time: 1.062 sec

Data scanned: 2.13 KB

Results (2)

Search rows

pricearea total_offshore_wind total_onshore_wind

1 DK1 1841.73999 777.819992

2 DK2 530.3400260000001 158.980004

The image is a composite of three screenshots from the Amazon Athena console. The top screenshot shows the 'Query editor' interface with a SQL query:

```
1 SELECT pricearea,
2    Avg(solarpower) AS avg_solar_power
3 FROM "transformdata"."lawfeeq_energy_project_transform"
4 GROUP BY pricearea
5 ORDER BY avg_solar_power DESC;
```

 The query status is 'Completed' with a run time of 1.457 sec and 2.13 KB of data scanned. The results table has two columns: 'pricearea' and 'avg_solar_power'. The results are:

#	pricearea	avg_solar_power
1	DK1	1508.3566893333334
2	DK2	525.424988

 The middle screenshot shows the 'Query result encryption' settings. It includes the 'Query result location' (s3://athenaquerydaec/), the 'Encrypt query results' checkbox (checked), the 'Encryption key' (arn:aws:kms:us-east-1:040838876532:key/61274d5c-a166-489c-a4b5-60481cf2b2d5), and the 'Expected bucket owner' (athenaquerydaec). The bottom screenshot shows the 'athenaquerydaec' console page. It displays a list of objects in an S3 bucket. The objects are:

Name	Type	Last modified	Size	Storage class
43ed610c-b65b-4ec8-8afc-32d7e49frc5d.csv	csv	July 31, 2025, 19:55:20 (UTC+08:00)	1.2 KB	Standard
43ed610c-b65b-4ec8-8afc-32d7e49frc5d.csv.metadata	metadata	July 31, 2025, 19:55:20 (UTC+08:00)	1.0 KB	Standard
91cc3806-a1d3-43f0-930c-884de76e53af.csv	csv	July 31, 2025, 19:55:00 (UTC+08:00)	1.2 KB	Standard

Justification: Amazon Athena is an ideal serverless SQL querying on S3 data and it eliminates the need for a relational database. It is cost efficient and can provide scalable queries even on large datasets. The transformed data was used for query as it is already in CSV format and structured accordingly for Athena Query. The queries were selected according to essential business insights such as comparing the energy production types.

Security Considerations: The data in the S3 bucket is encrypted at rest using AWS KMS.

Cost Considerations: Partitioning the data scanned reduced the cost and optimizes the query to scan only the relevant data.

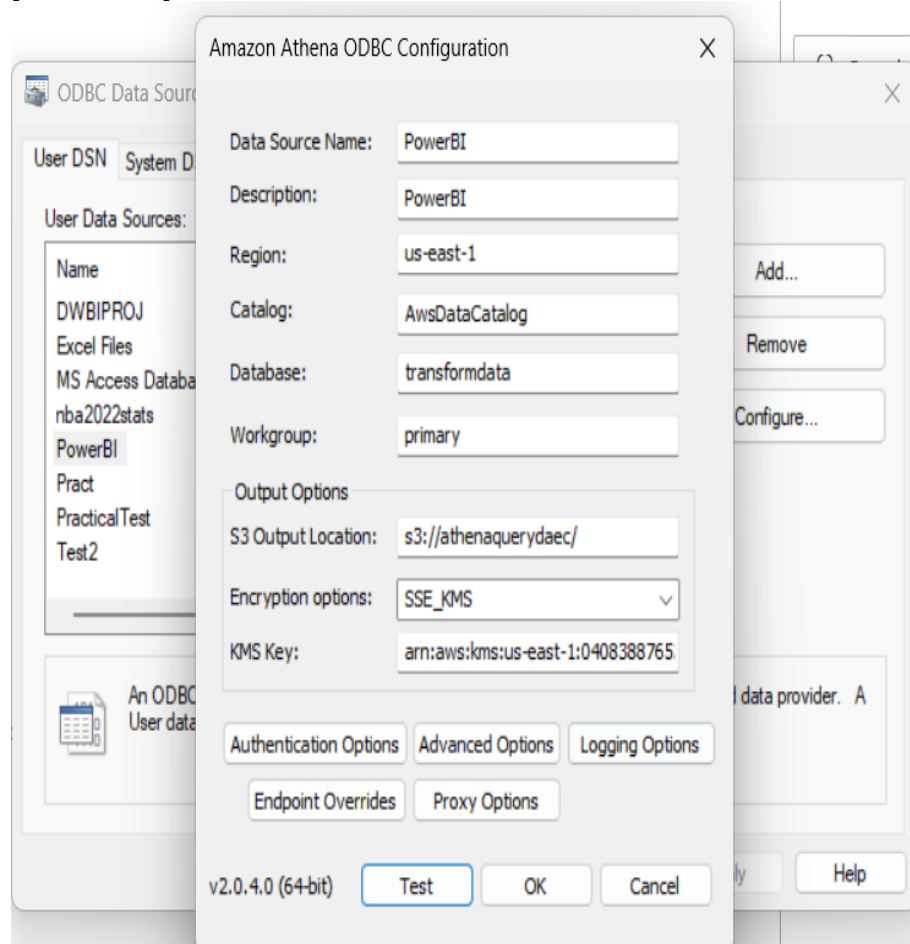
5. Data Analysis

ODBC
POWERBI
I

Setup: I used ODBC to connect Athena Query to Power BI. Amazon Athena ODBC was installed and registered in the ODBC Data source. I added the data source name and used this name to connect the ODBC to Power BI. The region, database, workgroup, s3 output location with the SSE_KMS KEY written down correctly and even the Access Key, Secret Access Key and Session Token was used to connect to my Athena in AWS Learner Lab. It was connected successfully, and I had to open up Power BI and type in all the credentials to access the data for analysis.

Additional Points: With ODBC, the data is queried directly from Amazon Athena which shows efficiency as it gets the data which is stored from S3. This further supports data integrity as there is a direct link to the data ensuring that the original data is not lost. Power BI allows for easy integration of large datasets as well, so it shows scalability. ODBC also provides real time access to the data so any updates made will be reflected immediately. The use of SSE_KMS adds an additional layer of security ensuring that the data is encrypted at both rest and transit.

[screenshot]



Authentication Options



Authentication Type:	IAM Credentials
Username:	ASIAQTARPZV2BYJOZPPA
Password:	••••••••••••••••••••
Session Token:	IQoJb3JpZ2luX2VjEKz//////////wEaCX

	Connection test Successfully connected to: Athena engine version 3 Region: us-east-1 Catalog: AwsDataCatalog Workgroup: primary Auth Type: IAM Credentials OK
PowerBI	The first analysis is Energy Production vs Price Area. It shows the count of price area by offshore and onshore wind power. The count of the offshore seems to be higher than the count of onshore. The higher offshore production count can also suggest that there can be greater investments in certain regions. Potentially, it can be influenced by geographical factors such as infrastructure investments.
	Count of pricearea by offshorewindpower Count of pricearea by onshorewindpower
PowerBI	The second analysis is the energy production over time using a line chart. The offshore and onshore wind power shows a downward trend while the solar power shows an increase. This suggests that solar energy production could have increased during this period possibly due to better sunlight conditions. The time trend analysis is useful for identifying any seasonal impacts on renewable energy production. The downward trend could be due to wind conditions during this period. The increase in solar power aligns with the longer daylight hours as there is improved sunlight conditions. Which is why solar power production is more sensitive compared to wind power.

	Sum of offshorewindpower, Sum of onshorewindpower and Sum of solarpower by minutes5utc Sum of offshorewindpower Sum of onshorewindpower Sum of solarpower Sum of offshorewindpower, Sum of onshorewindpower and Sum of sola... 2,000 1,500 1,000 500 0 2025-07-31T08:35:00 2025-07-31T08:40:00 2025-07-31T08:30:00 minutes5utc <table><thead><tr><th>minutes5utc</th><th>Sum of offshorewindpower</th><th>Sum of onshorewindpower</th><th>Sum of solarpower</th></tr></thead><tbody><tr><td>2025-07-31T08:35:00</td><td>450</td><td>150</td><td>2100</td></tr><tr><td>2025-07-31T08:40:00</td><td>450</td><td>150</td><td>2000</td></tr><tr><td>2025-07-31T08:30:00</td><td>300</td><td>100</td><td>1500</td></tr></tbody></table>	minutes5utc	Sum of offshorewindpower	Sum of onshorewindpower	Sum of solarpower	2025-07-31T08:35:00	450	150	2100	2025-07-31T08:40:00	450	150	2000	2025-07-31T08:30:00	300	100	1500
minutes5utc	Sum of offshorewindpower	Sum of onshorewindpower	Sum of solarpower														
2025-07-31T08:35:00	450	150	2100														
2025-07-31T08:40:00	450	150	2000														
2025-07-31T08:30:00	300	100	1500														
PowerBI	The third analysis is the Offshore vs Onshore Wind production shown by a scatter plot. Most of the offshore ranges from 150 to 200 while the onshore ranges from 40 to 60. This shows a clear difference between both wind powers. This also shows that offshore wind farms can be more efficient compared to onshore in the regions identified. The difference between both wind power can be impacted by local geographical features in each region. For example, DK2 can have access to stronger winds compared to DK1. For economy, offshore wind projects will have a higher investment cost and can make them more used in the long run.																

	The scatter plot displays the relationship between offshore wind power (x-axis) and onshore wind power (y-axis). The x-axis ranges from 100 to 350 with major ticks every 50 units. The y-axis ranges from 40 to 140 with major ticks every 20 units. There are two distinct clusters of data points, represented by blue dots. The first cluster is located at approximately (125, 40), indicating low offshore power and low onshore power. The second cluster is located at approximately (305, 130), indicating high offshore power and high onshore power. The plot area is overlaid with a light gray dashed grid.
	Justification: The line chart was used for energy production trends to provide insights on how energy will fluctuate over time, and it helped identify trends in offshore, onshore and solar power. This will help monitor any changes based off seasons and help to optimize energy production strategies. Offshore vs Onshore scatter plot shows the efficiency of each wind production. It suggests that offshore can be more consistent as compared to onshore. This can help them analyse which energy type to prioritise.

Advanced Requirements

6. Data Wrangling

Component	Setup: Jupyter Notebook was used to enhance the already transformed data and clean it and put it back into the same S3 bucket as a different file in CSV format. Using Boto3 library, I connect Amazon S3 and accessed the bucket. I first connected to the access key, secret key and session token to get access to my S3 buckets. From there, I got access to the file name containing the data to use it for further enhancement. To fill in the missing values, fillna() function. Once done, the data was formatted into a csv file and put back into the transformed S3 bucket as cleaned data.
-----------	--

[screenshot]

```
Jupyter DATAWRANGLER Last Checkpoint: yesterday
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

Requirement already satisfied: tzdata>=2022.1 in c:\users\tawfe\anaconda3\anaconda lab 01\lib\site-packages (from pandas) (2023.3)
Requirement already satisfied: urllib3<2.2.0,<3,>=1.25.4 in c:\users\tawfe\anaconda3\anaconda lab 01\lib\site-packages (from botocore<1.39.0,>=1.38.27-> boto3) (2.0.7)
Requirement already satisfied: six>=1.5 in c:\users\tawfe\anaconda3\anaconda lab 01\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

[5]: import boto3

session = boto3.Session(
    aws_access_key_id="ASTAQTPZVZKRSJOCUM",
    aws_secret_access_key="Spw1bFxp7G09Dn705dUuGwnSt57PQKClwb1C/v",
    aws_session_token="IQo7b3JpZ2ludXNjENP//////////wEaCXVzLXdlc3QlMjMhMEUCIQC0a8H977HsPDgYn3xcJNLzbgdFCVACcXcmp0bhjG58gIgm4ji2X1a66RV/g6y+YDcVW005Fd9w
    region_name="us-east-1" # Or your chosen region
)

s3 = session.client("s3")
response = s3.list_buckets()
for bucket in response["Buckets"]:
    print(bucket["Name"])

athenaquerydaec
aws-cloudtrail-logs-040838876532-9ae4144e
aws-glue-assets-040838876532-us-east-1
calabs-bucket-tawfeeq02
daec-test-2408212c
projecttawfeeqdaec02
tawfeeq-energy-project-2025
tawfeeq-energy-project-raw
tawfeeq-energy-project-transform

[10]: import pandas as pd
import io

bucket_name = 'tawfeeq-energy-project-transform'
object_key = 'run-AmazonS3_model753952034387-1-part-r-00000'

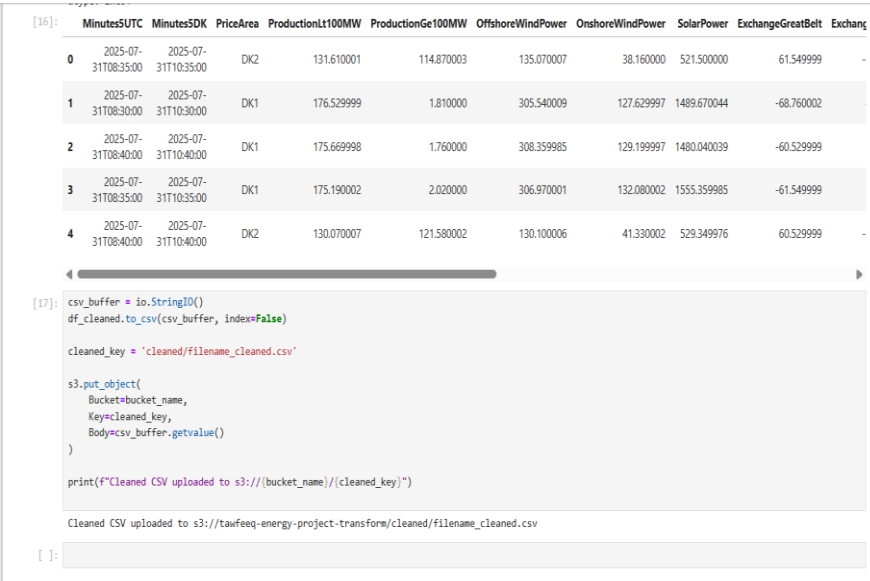
[16]: df = df.drop_duplicates()

# Drop rows with any missing values
df_cleaned = df.fillna(0)
print(df_cleaned.isnull().sum())
# OR: Fill with 0
# df_cleaned = df.fillna(0)

# OR: Forward-fill
# df_cleaned = df.fillna(method='ffill')

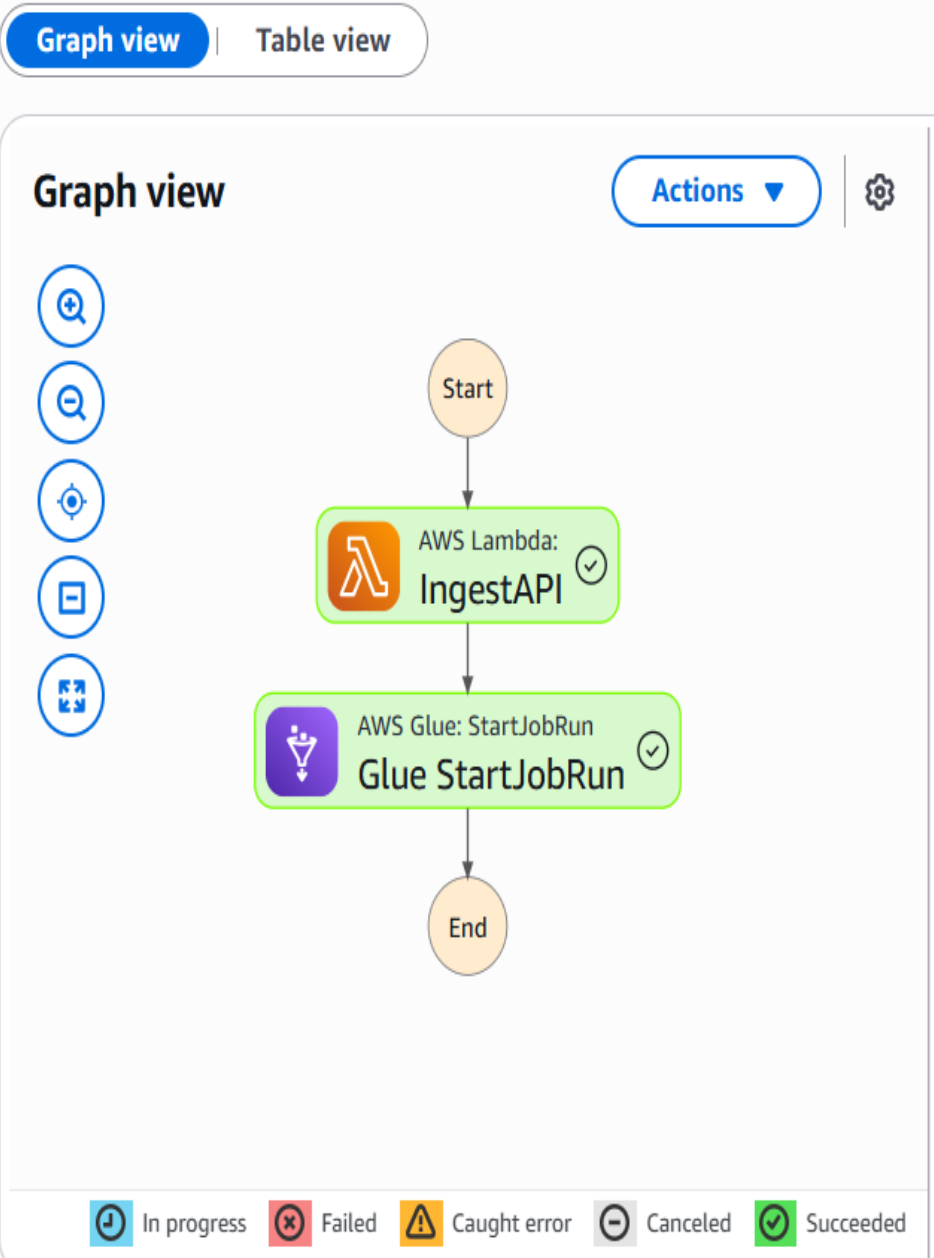
(df_cleaned.head())

MinutesUTC      0
Minutes5DK      0
PriceArea       0
ProductionLt100M 0
ProductionGe100M 0
OffshoreWindPower 0
OnshoreWindPower 0
SolarPower      0
ExchangeGreatBelt 0
ExchangeGermany 0
ExchangeNetherlands 0
ExchangeGreatBritain 0
ExchangeNorway 0
ExchangeSweden 0
BornholmSE4    0
partition_0     0
partition_1     0
partition_2     0
dtype: int64
```

	 The screenshot shows a Jupyter Notebook interface. At the top, a data table is displayed with columns: MinutesUTC, MinutesSDK, PriceArea, ProductionLt100MW, ProductionGe100MW, OffshoreWindPower, OnshoreWindPower, SolarPower, ExchangeGreatBelt, and Exchange. The table contains 5 rows of data. Below the table, a scroll bar is visible. Underneath the scroll bar, Python code is shown in a code cell, which includes importing io, converting a DataFrame to a CSV buffer, and using boto3 to upload the CSV to an S3 bucket. The code also includes a print statement to confirm the upload. The output of the code cell shows the file path where the CSV was uploaded.
	Justification: Using Jupyter Notebook and Boto3 allows the process to be automated and scalable. This is used to ensure the data is entirely cleaned and as new data is ingested, the cleaning process can be easily repeated. Handling the missing values are very essential for handling any data through wrangling as you can ensure that the dataset is complete and ensure that the missing values are input randomly and does not follow a specific pattern to prevent bias. Data integrity was also ensured in this process as we are preserving the original transformed data. This allows us to keep the data in case there should be any issues when wrangling the data, and it can be sent again for cleaning. Security was also used as I had to use IAM credentials such as Access Key, Secret Access Key and Session Token to access the S3 bucket. This ensures data security and governance. The wrangling process improves the quality of data and ensures that the insights from the data analysis were accurate and reliable.

7. Deployment/Orchestration

StepFunctions	Setup: Step Function was used to automate the data pipeline. It consists of two core steps. Firstly, the AWS Lambda Function that ingests the API from Energinet real time data. This will automate the ingestion of the data and store it into the raw JSON S3 bucket. Next, AWS Glue Job will remove any duplicates and transform the raw JSON data into CSV format and store it into the transformed S3 bucket for any query and analysis via Athena.
---------------	--

	[screenshot]  <pre> graph TD Start([Start]) --> IngestAPI[AWS Lambda: IngestAPI] IngestAPI --> GlueStartJobRun[AWS Glue: StartJobRun] GlueStartJobRun --> End([End]) </pre> The screenshot displays the AWS Step Functions console in 'Graph view'. At the top, there are tabs for 'Graph view' (selected) and 'Table view'. Below the tabs, the title 'Graph view' is shown next to an 'Actions' dropdown and a settings gear icon. On the left side, there is a vertical toolbar with icons for zooming in, zooming out, full screen, and other navigation tools. The main area shows a workflow diagram with three nodes: a yellow 'Start' node, a green 'AWS Lambda: IngestAPI' task node, and a green 'AWS Glue: StartJobRun' task node, all connected by downward arrows. The workflow ends at a yellow 'End' node. At the bottom, a legend indicates the status of tasks: 'In progress' (blue clock icon), 'Failed' (red X icon), 'Caught error' (yellow warning icon), 'Canceled' (gray minus icon), and 'Succeeded' (green checkmark icon).
	Justification: The Step Functions allows us to give a visual representation for the automated pipeline which makes it easy for scaling and debugging. For cost considerations, with only two transitions every hour, 720 transitions per month will only cost under \$1 per month showing it is very cost-efficient. Step Functions allows us to monitor the status of each task. If any task were to fail, the pipeline can alert via SNS.
Eventbridge	Setup: Amazon Event Bridge was configured to trigger the Step Function at a fixed rate of every 1 hour using schedules. This will initiate the Step Functions automatically every hour and removes the need for any manual execution and automates the process.

	DAECProjRule DAECProjRule Enable Edit Delete Schedule detail Schedule name DAECProjRule Description - Schedule group name default Status Disabled Schedule ARN arn:aws:scheduler:us-east-1:040838876532:schedule/default/DAECProjRule Action after completion NONE Schedule start time - Schedule end time - Execution time zone Asia/Singapore Flexible time window 1 hour Created date Aug 04, 2025, 10:00:33 (UTC+08:00) Last modified date Aug 05, 2025, 11:38:13 (UTC+08:00)
--	--

Schedule

Target

Retry policy

Dead-letter queue

Encryption

Schedule

Fixed rate

rate (1 hours)

8. Monitoring

Cloudwatch	Setup: An alarm was created for the ingestion API lambda function. It was set to have a threshold of whenever errors is greater/equal to 1. Then, an in alarm was set to send notification via SNS to the email in case of any alerts. Cloud Watch was also used to monitor the logs of critical services such as Lambda and Glue. This will give us an overview of the execution status and any potential failures and the runtime logs and job duration for lambda and glue respectively. These logs can be used for debugging in the event of any error.
	[screenshot] AlarmLambda The actions have been successfully enabled. Alarms (1) Search Alarm state: Any Alarm type: Any Actions status: Any Hide Auto Scaling alarms < 1 > AlarmLambda Metric alarm OK AlarmLambda View Actions Explore related Investigate Count 1 0.5 0 09:15 09:20 09:25 09:30 09:35 09:40 09:45 09:50 09:55 10:00 10:05 10:10 Errors Click timeline to see the state change at the selected time. 9:15 9:20 9:25 9:30 9:35 9:40 9:45 9:50 9:55 10:00 10:05 10:10 In alarm OK Insufficient data Disabled actions

	AlarmLambda > Edit Configure actions - optional Notification Alarm state trigger Define the alarm state that will trigger this action. ☒ In alarm The metric or expression is outside of the defined threshold. ☐ OK The metric or expression is within the defined threshold. ☐ Insufficient data The alarm has just started or not enough data is available. Remove Send a notification to the following SNS topic Define the SNS (Simple Notification Service) topic that will receive the notification. ☒ Select an existing SNS topic ☐ Create new topic ☐ Use topic ARN to notify other accounts Send a notification to... PipelineAlert Only topics belonging to this account are listed here. All persons and applications subscribed to the selected topic will receive notifications. Email (endpoints) 2400212C@student.tp.edu.sg - View in SNS Console Add notification CloudWatch > Log groups > /aws/lambda/ProjectFunctionDAEC > 2025/08/05/[\$LATEST]ce786a38ef474181901a45acd787dcc6 CloudWatch Log events You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns Filter events - press enter to search Clear 1m 30m 1h 12h Custom UTC timezone Display <table> <thead> <tr> <th>Timestamp</th> <th>Message</th> </tr> </thead> <tbody> <tr> <td colspan="2">No older events at this moment. Retry</td> </tr> <tr> <td>2025-08-04T01:51:20.417Z</td> <td>INIT_START Runtime Version: python:3.13.v50 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:83a0b29e480e14176225231a6e561282...</td> </tr> <tr> <td>2025-08-04T01:51:20.526Z</td> <td>START RequestId: 64649eb2-572d-4052-bd6f-31d41b764859 Version: \$LATEST</td> </tr> <tr> <td>2025-08-04T01:51:21.941Z</td> <td>Uploaded 5 records to s3://taufeeq-energy-project-raw/2025-08-04/01/51/electricity_data.json</td> </tr> <tr> <td>2025-08-04T01:51:21.951Z</td> <td>END RequestId: 64649eb2-572d-4052-bd6f-31d41b764859</td> </tr> <tr> <td>2025-08-04T01:51:21.952Z</td> <td>REPORT RequestId: 64649eb2-572d-4052-bd6f-31d41b764859 Duration: 1025.11 ms Billed Duration: 1026 ms Memory Size: 128 MB Max Memory...</td> </tr> <tr> <td colspan="2">No newer events at this moment. Auto retry paused. Resume</td> </tr> </tbody> </table>	Timestamp	Message	No older events at this moment. Retry		2025-08-04T01:51:20.417Z	INIT_START Runtime Version: python:3.13.v50 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:83a0b29e480e14176225231a6e561282...	2025-08-04T01:51:20.526Z	START RequestId: 64649eb2-572d-4052-bd6f-31d41b764859 Version: \$LATEST	2025-08-04T01:51:21.941Z	Uploaded 5 records to s3://taufeeq-energy-project-raw/2025-08-04/01/51/electricity_data.json	2025-08-04T01:51:21.951Z	END RequestId: 64649eb2-572d-4052-bd6f-31d41b764859	2025-08-04T01:51:21.952Z	REPORT RequestId: 64649eb2-572d-4052-bd6f-31d41b764859 Duration: 1025.11 ms Billed Duration: 1026 ms Memory Size: 128 MB Max Memory...	No newer events at this moment. Auto retry paused. Resume	
Timestamp	Message																
No older events at this moment. Retry																	
2025-08-04T01:51:20.417Z	INIT_START Runtime Version: python:3.13.v50 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:83a0b29e480e14176225231a6e561282...																
2025-08-04T01:51:20.526Z	START RequestId: 64649eb2-572d-4052-bd6f-31d41b764859 Version: \$LATEST																
2025-08-04T01:51:21.941Z	Uploaded 5 records to s3://taufeeq-energy-project-raw/2025-08-04/01/51/electricity_data.json																
2025-08-04T01:51:21.951Z	END RequestId: 64649eb2-572d-4052-bd6f-31d41b764859																
2025-08-04T01:51:21.952Z	REPORT RequestId: 64649eb2-572d-4052-bd6f-31d41b764859 Duration: 1025.11 ms Billed Duration: 1026 ms Memory Size: 128 MB Max Memory...																
No newer events at this moment. Auto retry paused. Resume																	
	Justification: Cloud watch is a service that integrates with other services like Lambda and Glue. It will provide real-time monitoring of the pipeline and allows performance optimization. As for the costs, basic monitoring for short durations are of low costs and there is no high frequency log scanning.																
SNS	Setup: An SNS topic called Pipeline Alert was created to alert the user. A subscription was made using email protocol and the email was input. This will give the user email notification for events such as service errors or through the use of publishing a message.																

	Simple Notification Service Subscription confirmed! You have successfully subscribed. Your subscription's id is: <code>arn:aws:sns:us-east-1:040838876532:PipelineAlert:adb39018-6663-41f0-b5e8-04fc209f58a6</code> If it was not your intention to subscribe, click here to unsubscribe.
	Justification: SNS is a scalable way to deliver notifications in real time scenario for events like pipeline failure or security issues. For costs, SNS has no cost for sending emails so no extra cost is used.
GuardDuty	Setup: Guard Duty was activated for any threat decision across the resources. It will monitor and unauthorized access or any unusual API calls. Any findings will be available via the Guard Duty console so not configuration is needed. Justification: Enabling Guard Duty adds an additional layer of protection and alerts over services such as S3, Lambda and so on. Similar to SNS, Guard Duty has no extra cost, making it cost-free and essential in terms of security.